

Question 1: Suppose a disk drive has 5000 cylinders numbered 0 to 4999. The drive currently serves a request at cylinder 2150; the previous request was at cylinder 1,805. The queue of pending requests, in FIFO order, is:
2069; 1212; 2296; 2800; 544; 1618; 356; 1523; 4965; 3681

For each disk-scheduling algorithm, determine the order in which the pending requests will be serviced, starting from the current head position.

- a) FCFS
- b) SCAN
- c) C-SCAN

Question 2: Given six memory partitions of 100 MB, 170 MB, 40 MB, 205 MB, 300 MB, and 185 MB (in order), how would the first-fit, best-fit, and worst-fit algorithms place processes of size 200 MB, 15 MB, 185 MB, 75 MB, 175 MB, and 80 MB (in order)? Indicate which requests cannot be satisfied (if any). Comment on how efficiently each of the algorithms manages memory.

Question 3: Consider a logical address space of 2,048 pages with a 4-KB page size, mapped onto a physical memory of 512 frames.

- a. How many bits are required in the logical address?
- b. How many bits are required in the physical address?

Question 4: Consider a system that uses paging. Virtual and physical addresses are 32 bits long, and the page size is $4KB = 2^{12}$ bytes. Process P1 has the following page table. Page and frame numbers are given in hexadecimal notation (recall that each hexadecimal digit represents 4 bits).

Page Number	Frame Number
0x00000	0x00234
0x00001	0x00235
0x00002	0x0023f
0x00003	0x00ace
0x00004	0x00bcd

For each virtual address, indicate the physical address to which it maps. If the virtual address is not part of the address space of P1, write “NO TRANSLATION” instead. Use hexadecimal notation for physical addresses. *Add your answers to the table below.*

- a. 0x00001a60
- b. 0x002351ff
- c. 0x00000fb5
- d. 0x00003002

Question 5: Consider the following page reference string: 7, 2, 3, 1, 2, 5, 3, 4. Assuming demand paging with three frames, how many page faults would occur for the following replacement algorithms?

- a) LRU replacement
- b) FIFO replacement

Good luck!